Project Report On:

# TPQA Management System

"Academic Consultancy & Third-Party Quality Audit Management Platform"

Submitted in partial fulfillment of the requirements
of
**B.Tech Information Technology**

By

Aditya Shimoga - 241080067
Taha Valiji - 241080071
Awwab Wadekar - 241080077
Yash Wadhwa - 241080078

Under the Guidance of
**Prof. V. B. Nikam**

**Computer Engg. & IT Department**
**Veermata Jijabai Technological Institute**
**Matunga, Mumbai – 400019**

# CONTENTS

1. Abstract.....3
2. Introduction.....4
3. Problem Statement.....5
4. Proposed Solution.....6
5. Tech Stack.....7
6. System Workflow and Operational Logic.....8
7. ER Diagram.....10
8. System Diagram.....12
9. System Screenshots.....15
10. Conclusion.....20

# 1. Abstract

The TPQA Management System is a full-stack web application developed to digitize and streamline the complete lifecycle of academic consultancy projects undertaken by university departments. Engineering institutions such as VJTI routinely engage with external organizations for consultancy, testing, and third-party quality audit work. Managing the end-to-end process, from the initial client request through approvals, invoicing, reporting, and revenue distribution. It involves significant manual effort across multiple stakeholders and departments.

The proposed platform automates this entire workflow through a structured 9-step pipeline, enforcing role-based access so that each stakeholder: Client Organizations, Faculty, HODs, Project Coordinators, and the Director, can only perform the actions relevant to their role at the appropriate stage. The system maintains a complete and immutable financial audit trail using strict database constraints, ensuring that invoices, receipts, and distribution records cannot be altered once finalized.

Built with a FastAPI backend, SQLAlchemy ORM, and a PostgreSQL database hosted on Supabase, the system provides a RESTful API secured with JWT authentication and bcrypt password hashing. The frontend, developed in Next.js 16 with TailwindCSS, offers separate dashboards for faculty and client users. Overall, the project demonstrates how a well-designed database-driven platform can replace error-prone manual processes in institutional consultancy management.

## 1.1 Domain

Database Management Systems, Academic Consultancy Lifecycle Automation and Institutional Financial Management.

## 2. Introduction

Academic institutions like Veermata Jijabai Technological Institute regularly undertake consultancy assignments for industry clients, ranging from technical audits and quality assessments to research and testing services. These engagements generate significant revenue that must be carefully tracked, invoiced, and distributed among the contributing faculty and departments. However, the existing approach to managing these projects is largely manual, relying on physical paperwork, email chains, and disconnected spreadsheets that are prone to delays, data loss, and audit difficulties.

The TPQA (Third-Party Quality Audit) Management System was conceived to replace this fragmented process with a unified digital platform. The system models the consultancy lifecycle as a strict 9-step workflow, ensuring that no stage can be skipped or completed out of order. Each step is tied to a specific role, and the platform enforces these rules programmatically, a client cannot approve their own budget, a support staff member cannot distribute funds, and a project cannot be closed before all financial records are finalized.

The platform serves five distinct user types: Client Organizations (external companies), Faculty Members (including Project Coordinators and HODs), the Director, and a Super Admin responsible for initial system setup. Each user type sees a tailored dashboard and can only interact with the workflow steps assigned to their role.

From a technical standpoint, the system is built as a decoupled full-stack application. The backend exposes a well-documented REST API using FastAPI (Python), while the frontend is a Next.js React application. The database layer uses PostgreSQL on Supabase with a carefully designed schema featuring 11 tables, 4 custom ENUM types, 2 analytics views, and auto-update triggers to maintain data integrity throughout the project lifecycle.

## 3. Problem Statement

Engineering departments in universities frequently engage in consultancy and third-party quality audit projects with external organizations. These projects involve multiple stages : client request, faculty response, budget approval, invoicing, payment collection, work completion, and revenue distribution, each requiring coordination between different stakeholders including client organizations, faculty members, department heads, and institutional directors.

In the absence of a dedicated system, this process suffers from several critical issues. Communication between stakeholders is unstructured and dependent on email or physical documents, making it difficult to track the current status of a project at any given time. Budget proposals, approvals, and financial records are maintained separately by different parties, creating discrepancies and making auditing difficult. There is no enforcement mechanism to ensure that steps are completed in the correct order or by the authorized person, leaving room for procedural errors.

Furthermore, revenue distribution, which must follow a fixed institutional split of 70% to the department and 30% to the institute, is calculated and tracked manually, increasing the risk of errors. The absence of a centralized record system means that historical project data, invoice histories, and distribution records are difficult to retrieve and analyze. These limitations reduce institutional efficiency, delay project closures, and undermine trust between the university and its industry partners.

## 4. Proposed Solution

To address these challenges, the TPQA Management System proposes a centralized, role-based web platform that digitizes the entire consultancy lifecycle. The core of the solution is a structured 9-step workflow that maps directly to the real-world stages of a consultancy project, enforced programmatically so that each step can only be executed by the appropriate role at the correct time.

The system provides separate login portals for Faculty (including Directors, HODs, and Project Coordinators) and Client Organizations, each with a tailored dashboard showing only the projects and actions relevant to them. JWT-based authentication ensures secure access, while bcrypt password hashing protects user credentials.

All financial records: budget estimations, proforma invoices, tax invoices, payment receipts, and revenue distribution entries, are stored in a relational PostgreSQL database with strict foreign key constraints marked ON DELETE RESTRICT, creating an immutable audit trail. CHECK constraints validate that distribution amounts sum correctly, preventing arithmetic errors in revenue splits.

Two analytic views: View_Institute_Cumulative_Revenue and View_Project_Coordinator_Analytics, provide institutional insights without requiring complex ad-hoc queries. Auto-update triggers on all tables ensure that Updated_At timestamps are always accurate, supporting audit and compliance requirements.

The frontend, built in Next.js with TailwindCSS, provides an intuitive workflow interface featuring a 9-step visual progress indicator, role-aware form rendering, and real-time status updates. The system is deployed with the backend on a cloud server and the frontend on Vercel, making it accessible from any device.

# 5. Tech Stack

The following table summarizes the technology choices made across all layers of the TPQA Management System:

| Layer | Technology | Version |
|---|---|---|
| Database | PostgreSQL (Supabase) | — |
| Backend | FastAPI (Python) | 0.115.12 |
| ORM | SQLAlchemy | 2.0.40 |
| Auth | python-jose (JWT) + passlib (Bcrypt) | — |
| Frontend | Next.js | 16.2.2 |
| Styling | TailwindCSS | 4.x |
| HTTP Client | Axios (patched CVE-2025-27152) | 1.7.9 |
| Testing | Pytest + httpx | — |

## 5.1 Backend

The backend is built using FastAPI, a modern Python web framework known for its high performance and automatic OpenAPI documentation generation. SQLAlchemy 2.0 is used as the ORM layer to interact with the PostgreSQL database, providing a Pythonic interface to all 11 database tables. The application is served by Uvicorn and organized into three routers — auth, consultancy, and admin — each handling a distinct domain of the API.

## 5.2 Database

PostgreSQL is used as the primary database, hosted on Supabase. The schema includes 11 tables, 4 custom ENUM types, 2 analytics views, and auto-update triggers on all tables. Supabase provides a managed PostgreSQL instance with built-in authentication helpers and a web-based SQL editor for schema management.

## 5.3 Authentication & Security

User authentication is implemented using JWT tokens signed with HS256 via python-jose. Passwords are hashed using bcrypt through the passlib library. CORS is configured to allow only trusted origins, and the Axios HTTP client on the frontend is pinned to version 1.7.9, patched against CVE-2025-27152.

## 5.4 Frontend

The frontend is developed using Next.js 16 with the App Router, styled with TailwindCSS 4.x. It communicates with the backend via Axios with JWT interceptors. The application features distinct dashboards for college faculty and client organizations, a 9-step visual workflow component, and dynamic modals for each workflow step.

## 6. System Workflow and Operational Logic

The TPQA system models the consultancy project lifecycle as a strict sequential 9-step workflow. Each step corresponds to a specific project status stored in the database as a custom ENUM, and each step can only be triggered by the authorized user role. The workflow enforces forward progression by default, with a controlled revert mechanism available for corrections.

| Step | Status | Action | Who |
|---|---|---|---|
| 1 | REQUEST_BY_EXTERNAL_ORG | Organization submits consultancy request | Client |
| 2 | RESPONSE_BY_FACULTY | Faculty assigns coordinator, proposes budget | Faculty |
| 3 | AGENCY_ACCEPTANCE | Client accepts proposed fee structure | Client |
| 4 | DIRECTOR_APPROVAL | Director reviews and approves budget | Director |
| 5 | PROFORMA_INVOICE | Generate proforma invoice | Faculty |
| 6 | TAX_INVOICE_AND_RECEIPT | Issue tax invoice, record payment receipt | Faculty |
| 7 | COMPLETION_REPORTS | Upload completion reports to client | Faculty |
| 8 | AMOUNT_DISTRIBUTION | Execute 70/30 revenue distribution | Faculty |
| 9 | CLOSED | Project sealed and archived | System |

### 6.1 Role-Based Access

The platform enforces role-based access at the API level. Every protected endpoint verifies the JWT token and resolves the current user's role before processing any request. The five user roles and their dashboard access are as follows:

| Role | Login Page | Permissions |
|---|---|---|
| Director | /login/faculty | Approve budgets (Step 4), view all projects |
| HOD | /login/faculty | Respond to requests (Step 2), view dept projects |
| Project Coordinator | /login/faculty | Full workflow management (Steps 2, 5–8) |
| Support Staff | /login/faculty | View projects only |
| Client / Organization | /login/client | Submit requests (Step 1), accept fees (Step 3) |

### 6.2 Workflow Step Details

Step 1 — Request by External Organization: The client organization initiates a consultancy request by submitting project details including the nature of work, expected duration, and department. This creates a new Project record in the database with status REQUEST_BY_EXTERNAL_ORG.

Step 2 — Faculty Response: An HOD or Project Coordinator reviews the request, assigns a coordinator, and proposes a budget estimation. The budget is stored in the Budget_Estimations table and the project status advances to RESPONSE_BY_FACULTY.

Step 3 — Agency Acceptance: The client reviews the proposed fee structure and either accepts or requests revision. Upon acceptance, the project moves to AGENCY_ACCEPTANCE status.

Step 4 — Director Approval: The Director reviews and approves the budget. This is the final approval gate before financial documents can be generated. Status advances to DIRECTOR_APPROVAL.

Steps 5 & 6 — Invoicing and Payment: A proforma invoice is generated first, followed by the official tax invoice once payment is received. The receipt is recorded in the Receipts table with payment details.

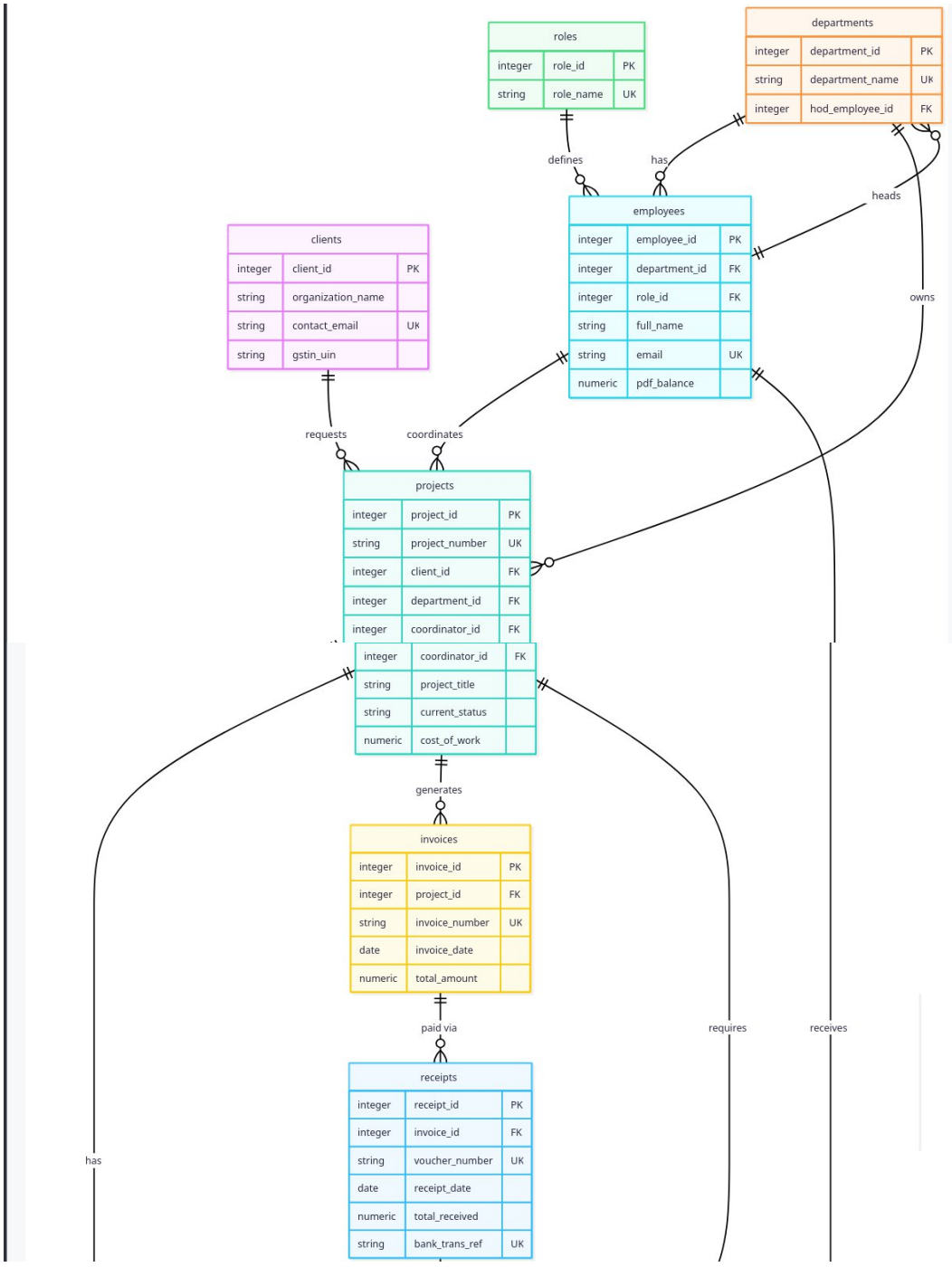
Step 7 — Completion Reports: The Project Coordinator uploads completion reports confirming that the consultancy work has been delivered to the client's satisfaction.

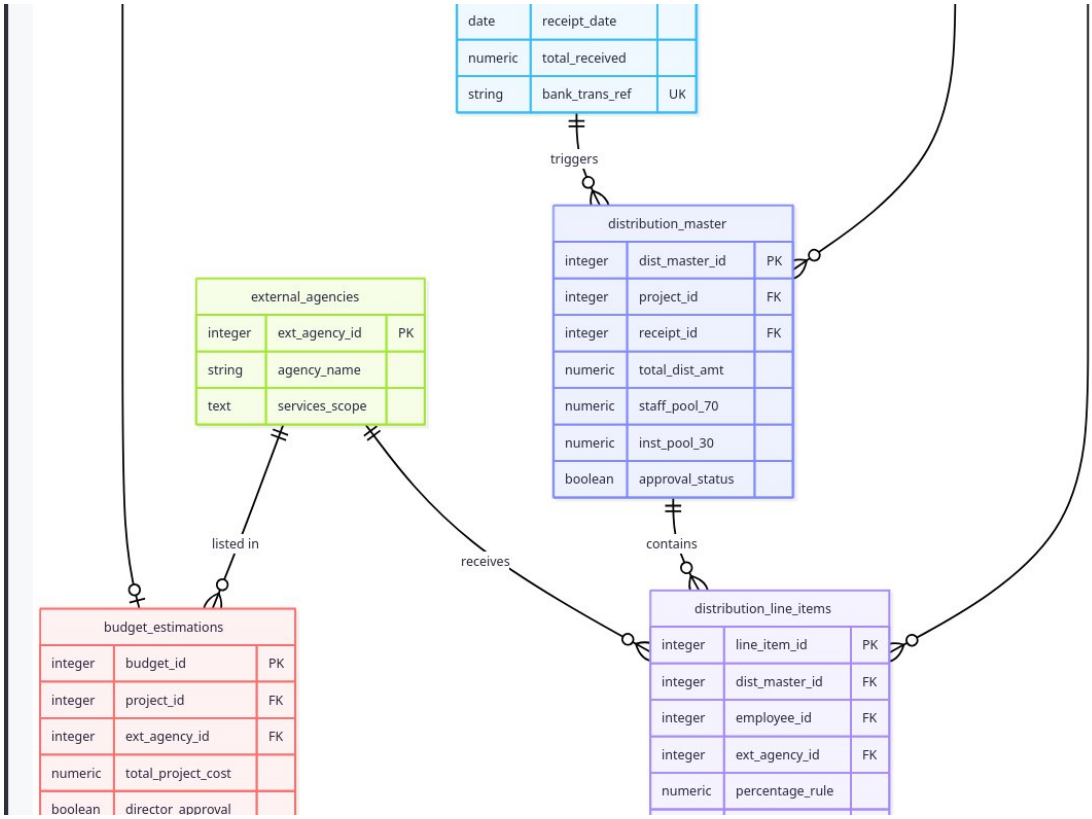
Step 8 — Amount Distribution: Revenue is distributed following the institutional 70/30 split. Distribution records are created in the Distribution_Master and Distribution_Line_Items tables with CHECK constraints validating the totals.

Step 9 — Project Closure: The project is sealed and archived. No further modifications are permitted after this step.

# 7. ER Diagram

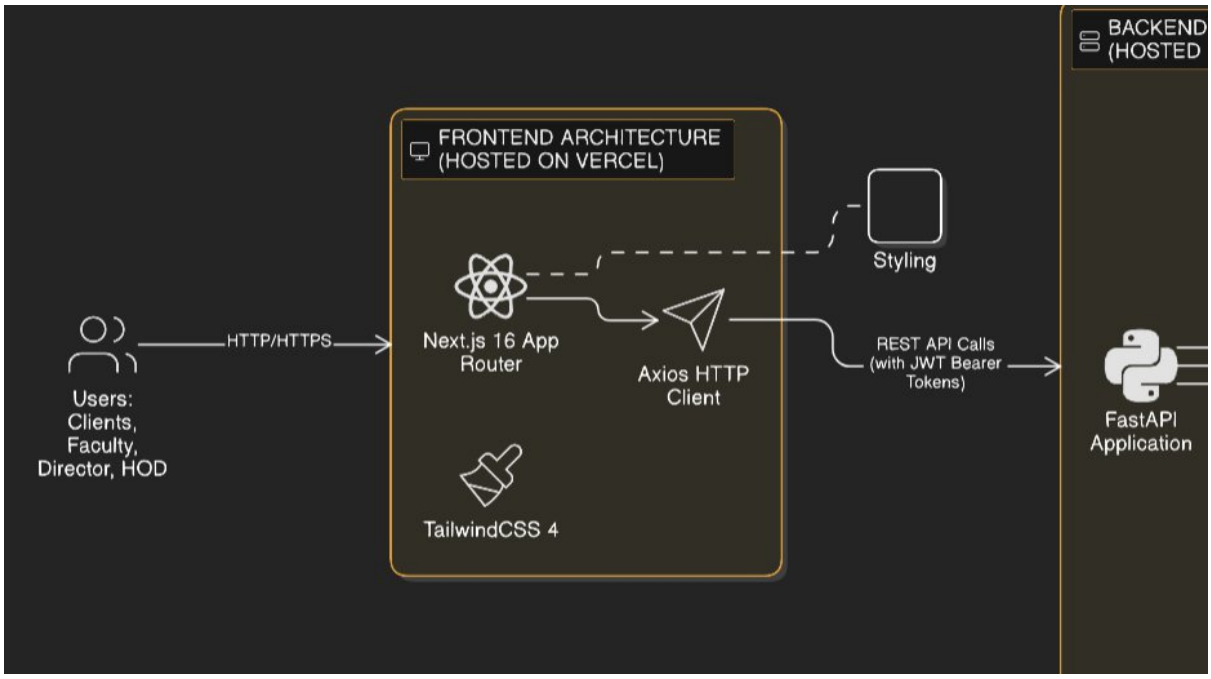
The Entity-Relationship (ER) diagram below illustrates the complete database schema of the TPQA Management System. The schema contains 11 tables: Roles, Departments, Employees, Clients, External_Agencies, Projects, Budget_Estimations, Invoices, Receipts, Distribution_Master, and Distribution_Line_Items. All financial foreign keys are marked ON DELETE RESTRICT to enforce an immutable audit trail.

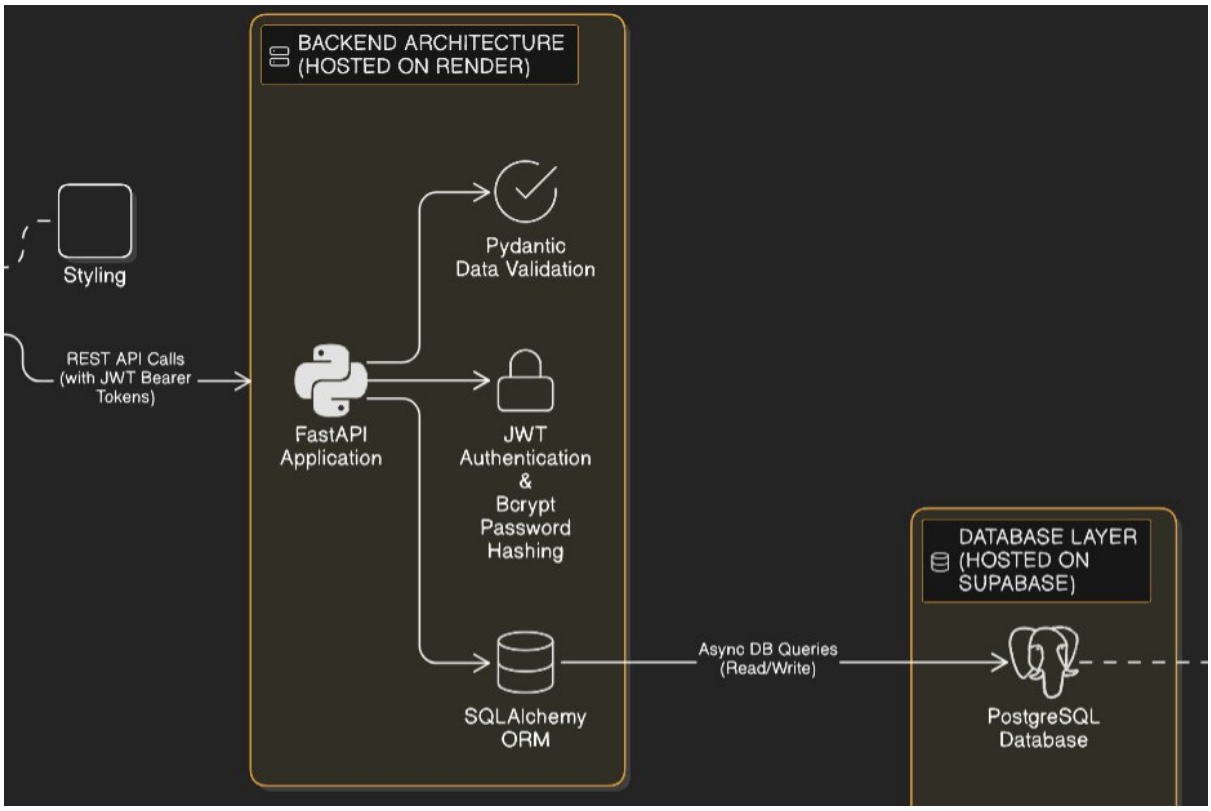


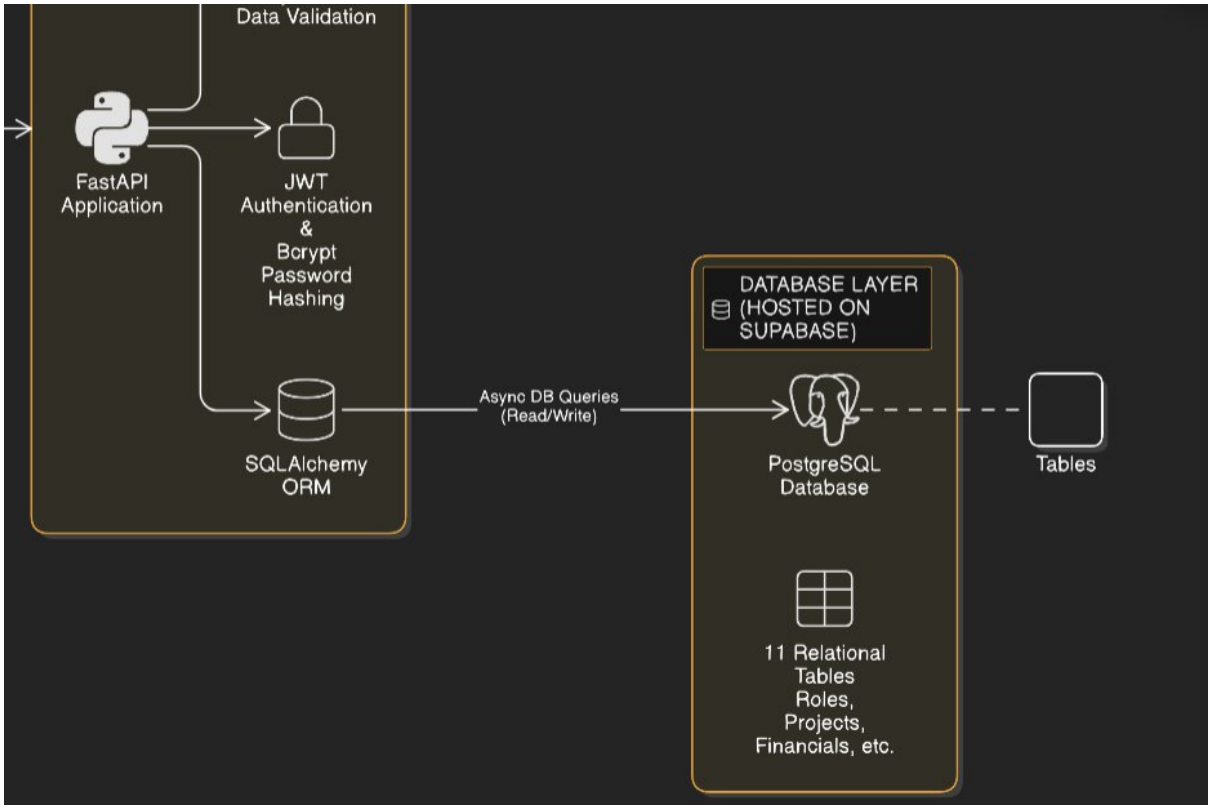

date receipt_date
numeric total_received
string bank_trans_ref UK
triggers
distribution_master
integer dist_master_id PK
integer project_id FK
integer receipt_id FK
numeric total_dist_amt
numeric staff_pool_70
numeric inst_pool_30
boolean approval_status
external_agencies
integer ext_agency_id PK
string agency_name
text services_scope
listed in
receives
contains
budget_estimations
integer budget_id PK
integer project_id FK
integer ext_agency_id FK
numeric total_project_cost
boolean director_approval
distribution_line_items
integer line_item_id PK
integer dist_master_id FK
integer employee_id FK
integer ext_agency_id FK
numeric percentage_rule

## 8. System Diagram

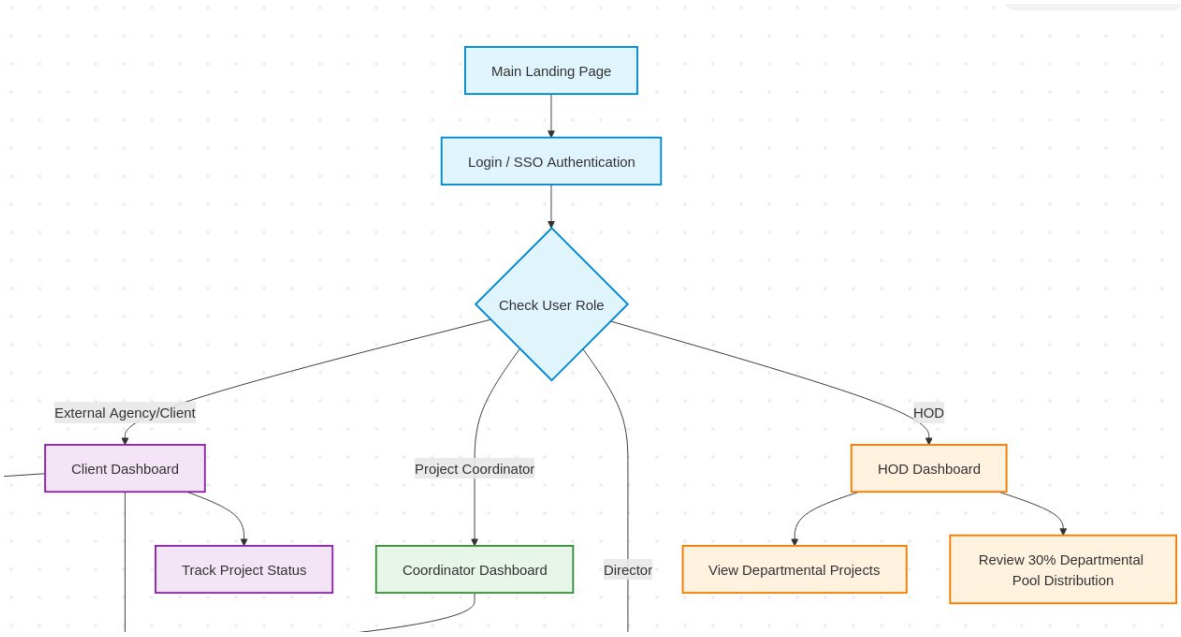
The architecture diagram below illustrates the high-level system design of the TPQA Management System, showing the interaction between the Next.js frontend, the FastAPI backend, the PostgreSQL database on Supabase, and the authentication layer.

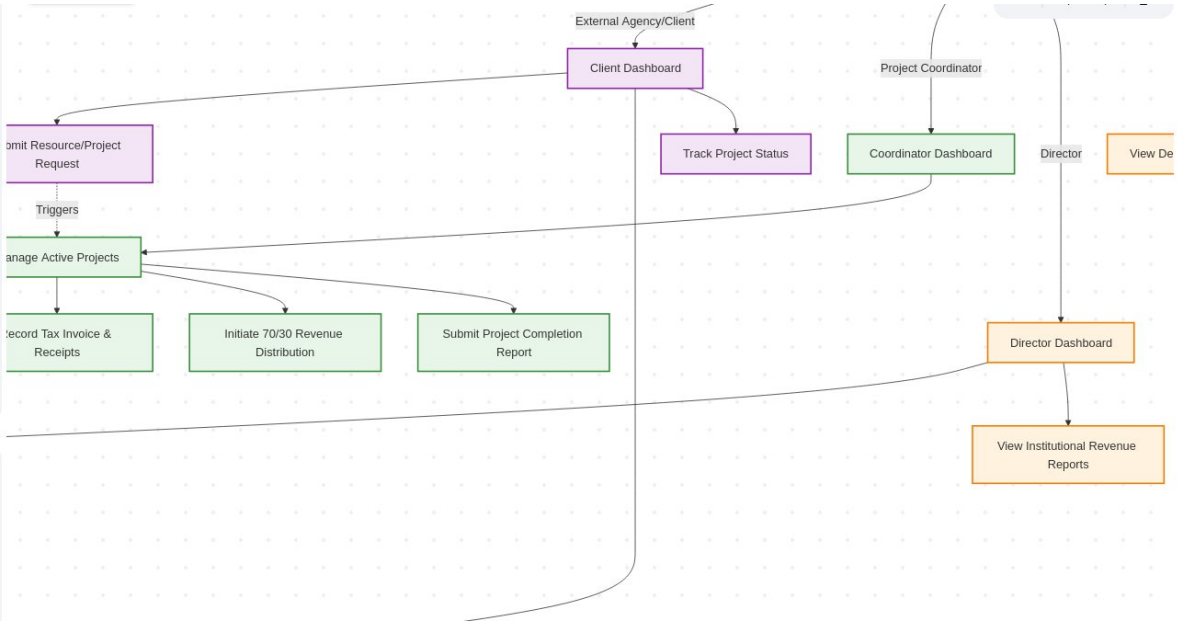


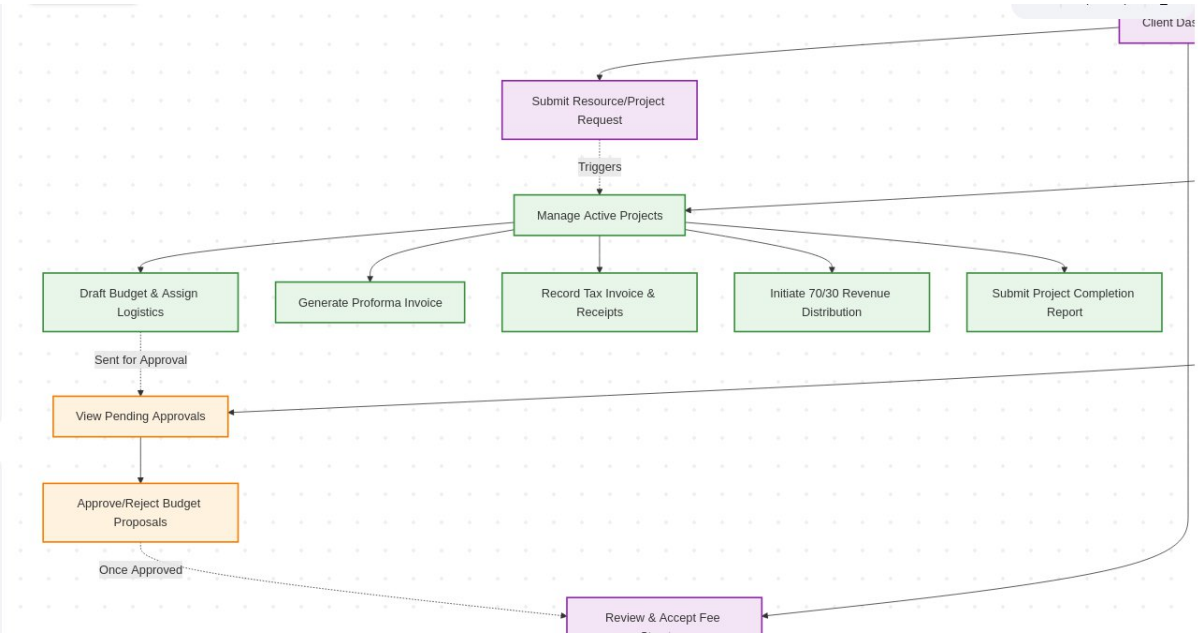





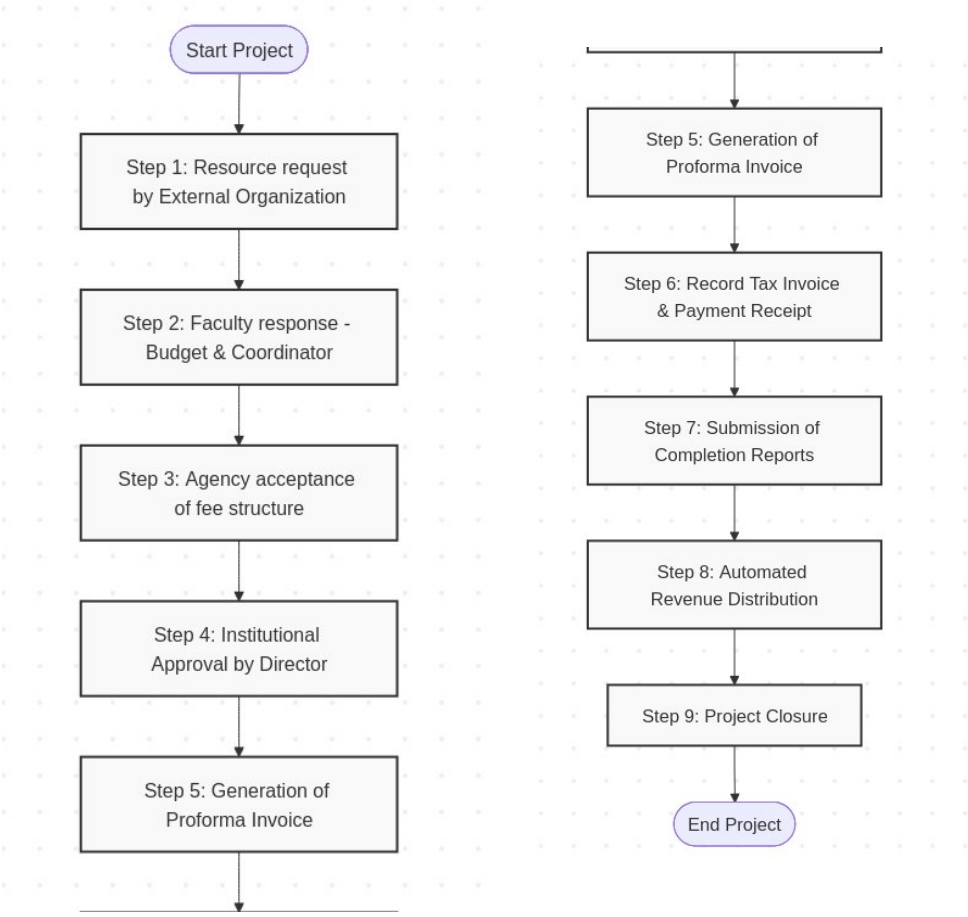

The sequence diagram below shows a typical end-to-end project flow, from client login and request submission through all nine workflow steps to final project closure.





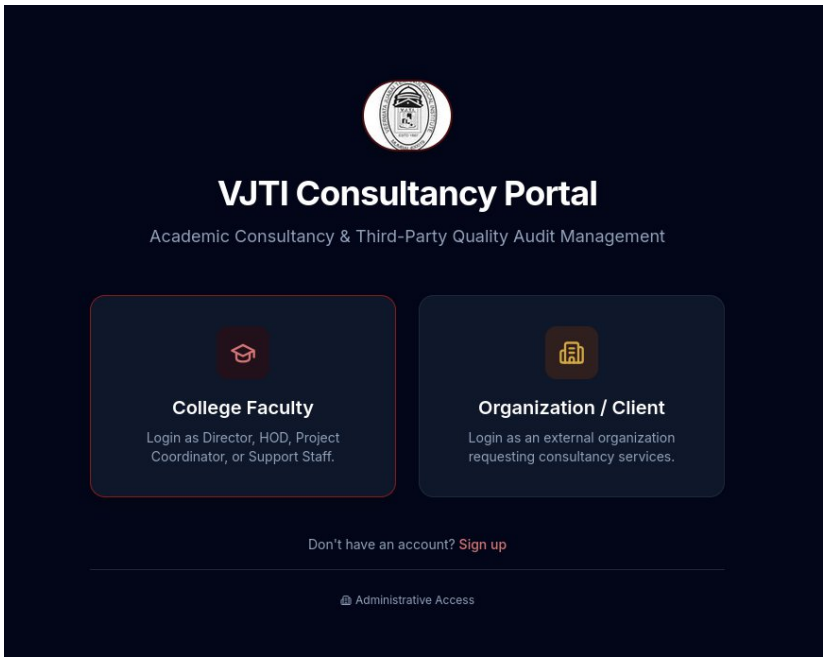




While this sequence diagram below shows a simplistic version focusing on the main 9 step pipeline.

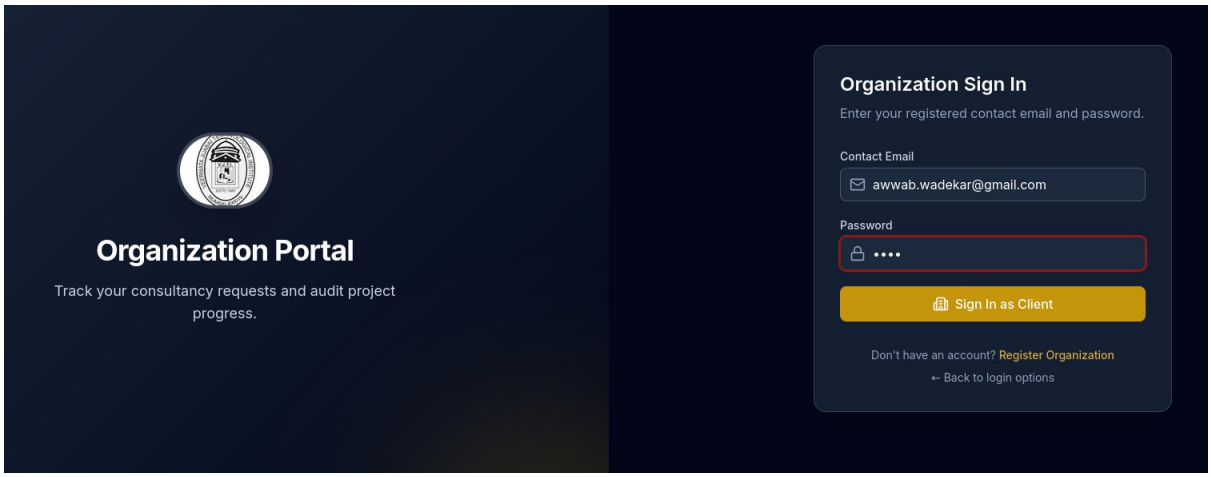

Start Project
Step 1: Resource request by External Organization
Step 2: Faculty response - Budget & Coordinator
Step 3: Agency acceptance of fee structure
Step 4: Institutional Approval by Director
Step 5: Generation of Proforma Invoice
Step 5: Generation of Proforma Invoice
Step 6: Record Tax Invoice & Payment Receipt
Step 7: Submission of Completion Reports
Step 8: Automated Revenue Distribution
Step 9: Project Closure
End Project

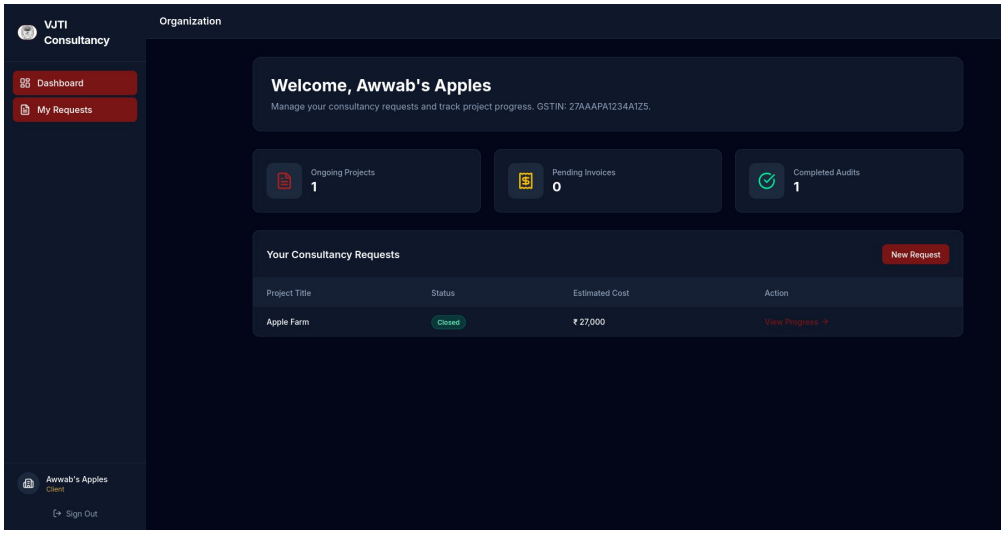
## 9. System Screenshots

The following screenshots illustrate the key screens of the TPQA Management System application.

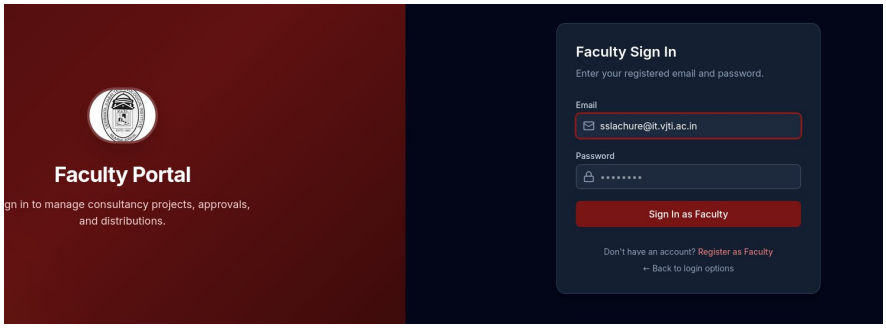


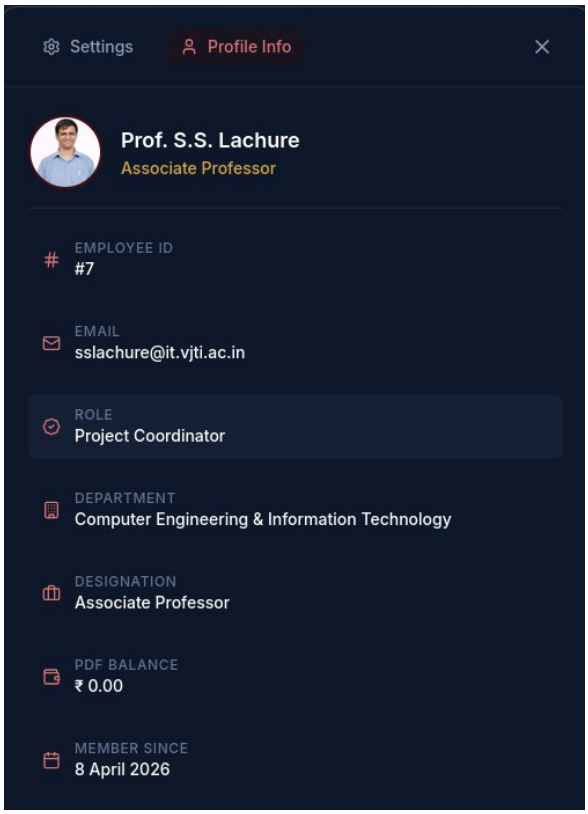

Client View





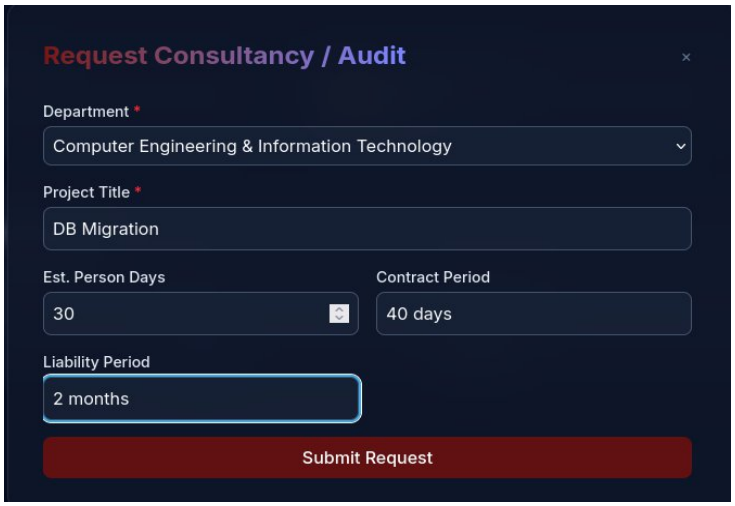

Faculty View:

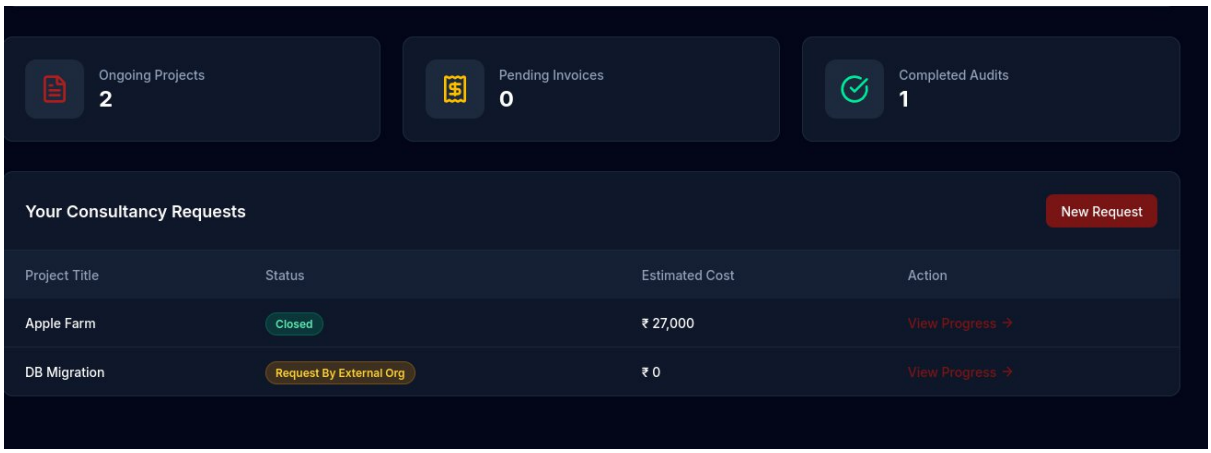


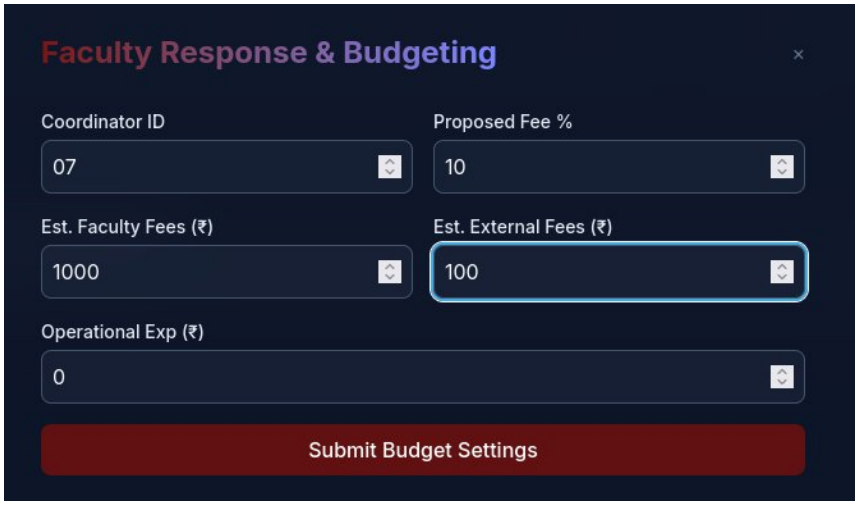



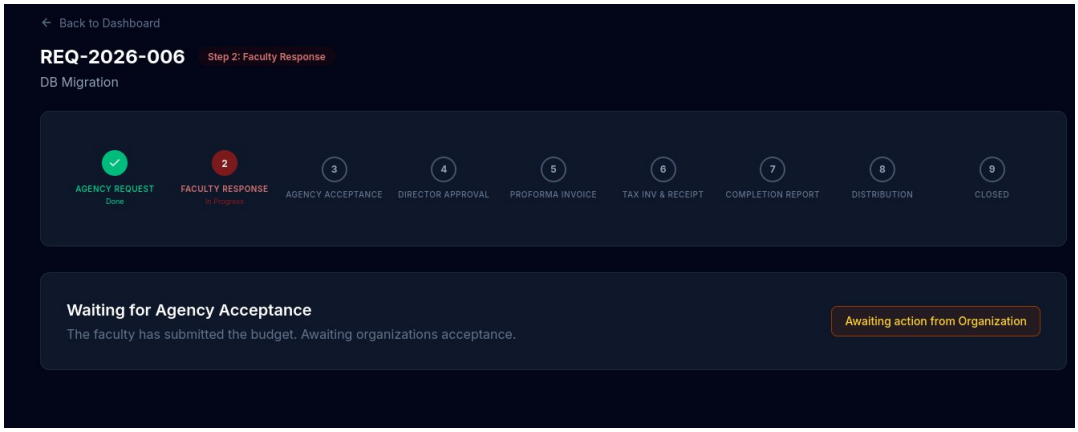
| | | | | |
|---|---|---|---|---|
| REQ-2026-001 | Apple Farm | Closed | ₹ 27,000 | View Details → |
| REQ-2026-002 | CAD Application downloads on Office PCs | Closed | ₹ 11,100 | View Details → |
| REQ-2026-003 | Calculations for strong bridge | Closed | ₹ 6,500 | View Details → |
| REQ-2026-004 | Calculating Material Costs Optimal | Closed | ₹ 6,000 | View Details → |

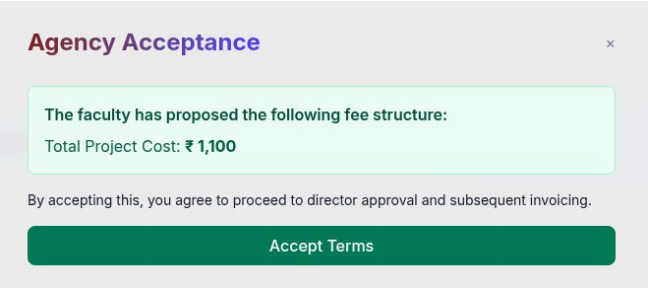
Some Sample SS of Process:

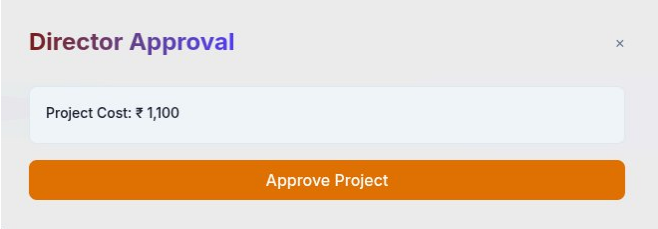


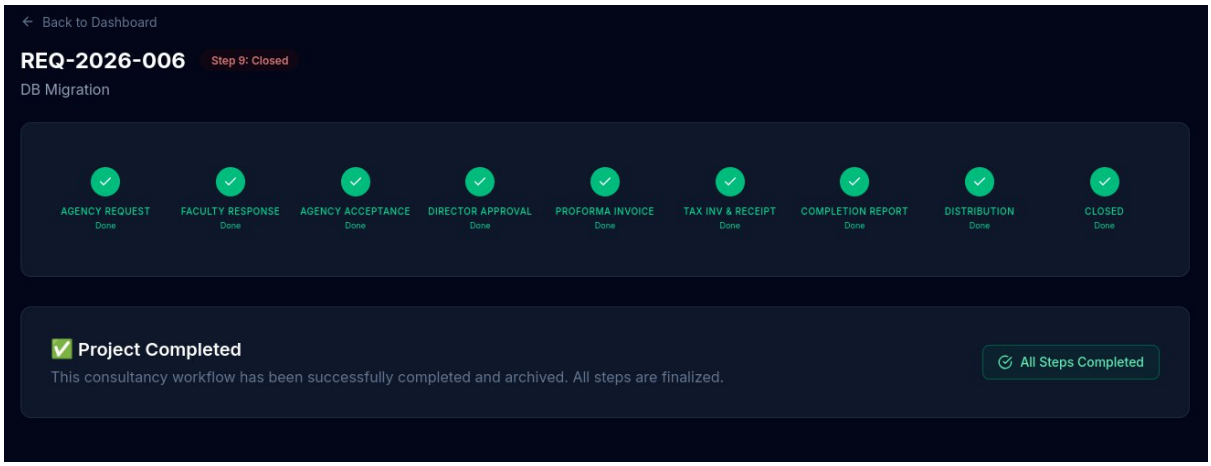




Back to Dashboard
REQ-2026-006
Step 2: Faculty Response
DB Migration
AGENCY REQUEST
Done
2
FACULTY RESPONSE
In Progress
3
AGENCY ACCEPTANCE
4
DIRECTOR APPROVAL
5
PROFORMA INVOICE
6
TAX INV & RECEIPT
7
COMPLETION REPORT
8
DISTRIBUTION
9
CLOSED
Waiting for Agency Acceptance
The faculty has submitted the budget. Awaiting organizations acceptance.
Awaiting action from Organization


Agency Acceptance
The faculty has proposed the following fee structure:
Total Project Cost: ₹ 1,100
By accepting this, you agree to proceed to director approval and subsequent invoicing.
Accept Terms


Director Approval
Project Cost: ₹ 1,100
Approve Project


Back to Dashboard
REQ-2026-006
Step 9: Closed
DB Migration
AGENCY REQUEST
Done
FACULTY RESPONSE
Done
AGENCY ACCEPTANCE
Done
DIRECTOR APPROVAL
Done
PROFORMA INVOICE
Done
TAX INV & RECEIPT
Done
COMPLETION REPORT
Done
DISTRIBUTION
Done
CLOSED
Done
✅ Project Completed
This consultancy workflow has been successfully completed and archived. All steps are finalized.
All Steps Completed

## 10. Conclusion

The TPQA Management System successfully digitizes the end-to-end lifecycle of academic consultancy projects, replacing a fragmented manual process with a structured, role-enforced digital workflow. By modelling the consultancy process as a strict 9-step pipeline and enforcing it programmatically through role-based access control and database-level constraints, the system ensures procedural correctness, financial integrity, and a complete audit trail.

The system's PostgreSQL schema — with its immutable financial foreign keys, CHECK constraints on distribution totals, and auto-update triggers — provides a robust foundation for institutional record-keeping. The two analytics views enable administrators and coordinators to derive revenue and performance insights without complex queries. The FastAPI backend's automatic Swagger documentation further makes the API easy to test and extend.

From a software engineering perspective, the project demonstrates clean separation of concerns between the router layer, business logic handlers, ORM models, and Pydantic schemas. The decoupled frontend-backend architecture allows each layer to be developed, tested, and deployed independently.

Future enhancements could include email notifications at each workflow stage, PDF generation for invoices directly within the system, granular role-based filtering of project lists, and a mobile-responsive progressive web app interface for field use.

For a detailed walkthrough of the architecture, data strategy, and API specifications, refer to the resources below:

Detailed Documentation: Google Drive – Full Documentation